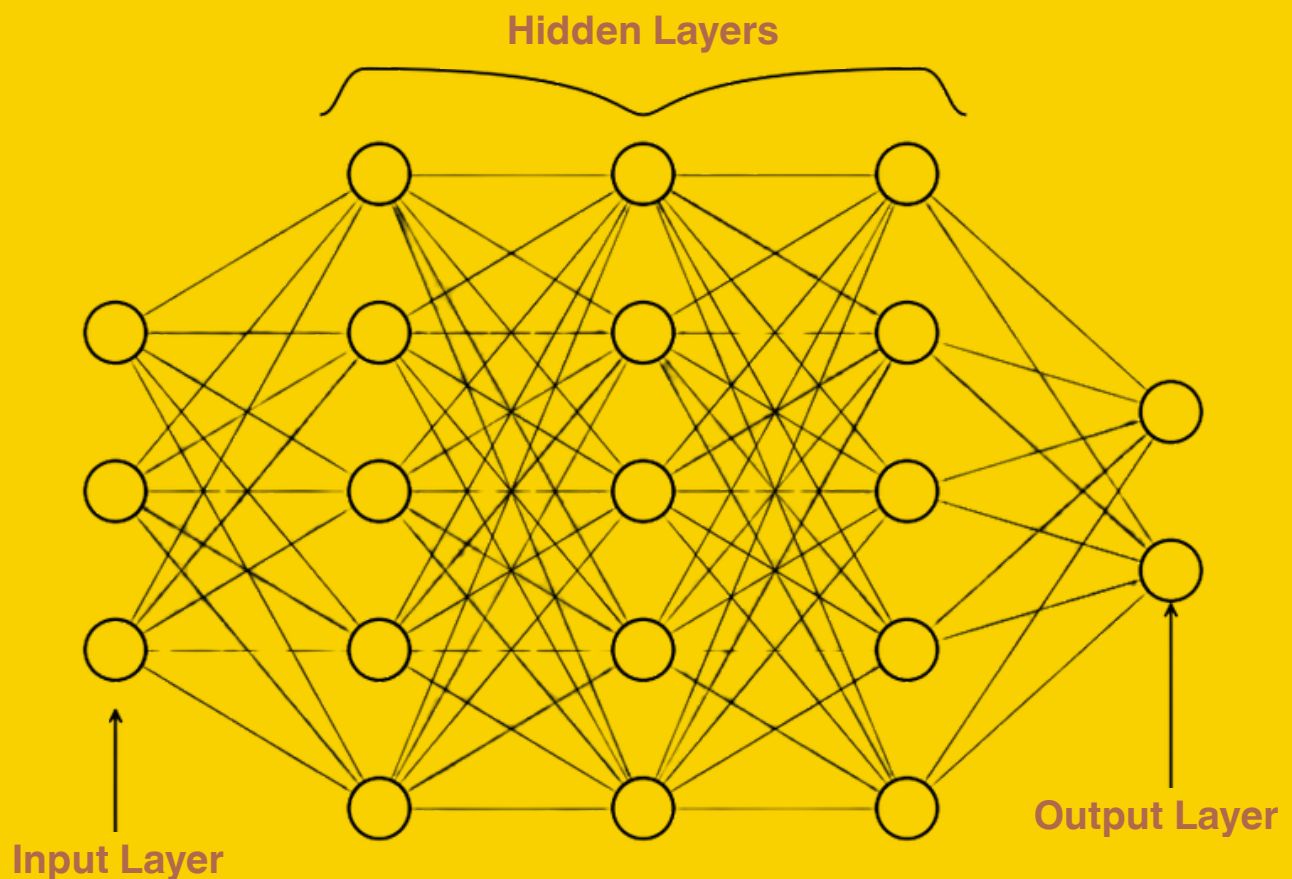


Modules for Machine and Deep Learning

Module 2: Multi-Layer Neural Network (with PyTorch)



Assoc. Prof. Ir. Dr. Airil Yasreen Mohd Yassin

School of Energy, Geoscience, Infrastructure and Society, Heriot-Watt University Malaysia

Assoc. Prof. Ts. Dr. Ahmad Razin Zainal Abidin

Faculty of Civil Engineering, Universiti Teknologi Malaysia

Dr. Halinawati Hirol

Malaysia-Japan International Institute of Technology (MJIT), Universiti Teknologi Malaysia

Dr. Mokhtazul Haizad Mokhtaram

Faculty of Engineering and Life Sciences, Universiti Selangor

Dr. Mohd Al-Akhbar Mohd Noor

Faculty of Engineering and Life Sciences, Universiti Selangor

Draft version
Not for distribution without
authors' permission

Table of Contents

1.	Introduction	1
2.	What We Will Implement in Module 2 That We Did Not Do in Module 1	1
	- Working with large data and files (CSV, Excel) - Data Splitting - Employ normalization or standardization - Working with libraries	
3.	PyTorch Workflow	3
4.	PyTorch Implementation (on Google Colab)	5
	Step 1: Import libraries	
	Step 2: Upload CSV file	
	Step 3: Read and inspect data	
	Step 4: Define features and targets	
	Step 5: Data splitting	
	Step 6: Normalization	
	Step 7: Convert to PyTorch tensors	
	Step 8: Build Neural Network	
	Step 9: Define Loss function	
	Step 10: Define optimizer	
	Step 11: Training	
	Step 12: Prediction using validation data	

1. Introduction

In Module 1 (or the Green Module), our objective was to demystify the “maths” of AI, specifically Neural Network (NN). To achieve this, we worked on a single-hidden-layer NN in representing a simple simultaneous equation:

$$2x + 2y = s \quad (1)$$

$$2x + 6y = t \quad (2)$$

There were two reasons for using the equations above:

1. To generate labelled data impromptu during workshops or classes. Otherwise, the typical pedagogy of starting with sets of gauged or measured data at the introductory level might distract us from the main objective of the workshop, that is, to understand and trace the “maths” behind NN.
2. To highlight the mathematical nature of NN. That is, NN is a universal approximator and can mimic equivalent mathematical relationships, either relationships that are “hidden” within the data or those that can be derived from physical conservation laws.

Also, in Module 1, we have written the MATLAB code from scratch, without using any AI libraries, as we want to establish the first-principles understanding of the “maths” and how they are implemented in the code. This is important to avoid the “black box” application of AI. We believe that, with such a first-principles understanding, it will:

- i. support the correct and effective use of AI technologies
- ii. speed up the learning of other AI concepts and models
- iii. motivate and support research and innovations in the field of AI, especially in engineering

Now that we have achieved the objective, in this Module 2, we will extend the discussion to the implementation of PyTorch. Our objective in this module is to demonstrate how NN can be implemented more effectively and properly (closer to industrial norms) with the use of available libraries.

However, we will stick with the philosophy that a new topic must be built upon the previous ones. So, to avoid an abrupt jump in the discussion, we will ensure a bridge between Module 1 and Module 2 by first discussing what we haven’t discussed and done in Module 1 but will do in Module 2.

2. What We Will Implement in Module 2 That We Did Not Do in Module 1

Below are some of the important points (differences) which we must highlight that can help us in reading Module 2.

1. Working with large data and files (CSV, Excel)

In common ML practice, we will work with very large data, stored or shared in files such as CSV and Excel. These data can be gauged or collected data obtained through long and laborious data collection processes, and/or publicly shared on platforms such as Kaggle, UCI, etc. Therefore, to work in ML or DL, we must familiarize ourselves with working with such files.

However, since we are still keeping the momentum of trying to understand NN mathematically, our labelled data will still be generated from simultaneous equations (we make them more complicated though). It is just that, this time, we will generate the set of inputs and outputs beforehand and store them in a CSV file. The equations used to generate our data are as follows:

$$3x + 0.2y + 2z^2 + 10 \sin(z) = s \quad (3)$$

$$0.05x^2 + 0.5y + 3z + 20\sin\left(\frac{x}{10}\right) = t \quad (4)$$

In the file shared, NN_simultaneous_equations.csv, 1000 labelled data/samples have been generated from these equations. In our Python environment (i.e., Google Colab), we will first upload this data before it is read by the code.

2. Data Splitting

In Module 1, we did mention that in ML, data splitting is required so that the model which has been trained can be tested and validated using “unseen data”. In actual practice, data splitting is usually carried out early in the ML workflow, before performing any data preprocessing that may learn information from the whole dataset. This is important to avoid data leakage. However, due to the introductory purpose of our discussion and module, we will carry out the data splitting immediately after the file is read.

In this module, we will simply split our data into 70% training data and 30% validation data. The training data will be used to train our NN, while the validation data will act as the “unseen data” to evaluate the prediction of the trained NN. Since we will not perform any hyperparameter tuning in this module, this simple splitting is sufficient for our current purpose. In a more complete ML workflow, particularly when hyperparameter tuning is involved, the data are commonly separated into training, validation and testing sets, which we will discuss in a later module.

3. Employ normalization or standardization

In Module 1, we have restricted the magnitude of our input to be small, where in the MATLAB code, they have been specified to range from 0 to 1. Such a restriction is needed because we didn't discuss or implement any scaling to our data.

In ML, data scaling is very crucial because we don't want variables with large values to dominate the training and affect the learning process. Two common scaling methods are normalization and standardization. In the former, we scale the data so that the magnitudes would typically be in the range of 0 to 1 (or sometimes -1 to 1). In the latter, we scale the data so that it has a mean of 0 and a standard deviation of 1.

In Module 2, we will implement normalization for both our features (inputs) and targets (outputs). Therefore, unlike in Module 1, we do not have to purposely restrict our raw data to small magnitudes. After the prediction, the predicted outputs will then be scaled back to their original magnitudes.

Due to this, the following are the ranges of values that we prescribed during the generation of the data in NN_simultaneous_equations.csv:

$$10 \leq x \leq 100$$

$$100 \leq y \leq 1000$$

$$1 \leq z \leq 10$$

4. Working with libraries

This is maybe the most important “take-home” from this module. The development of AI models, either for industrial applications or research, has been accelerated by the advancement of AI libraries. While in Module 1 we wrote everything from scratch, in practice we will employ various libraries that can ease and expand our works.

It is the main objective of this module to introduce some of the main libraries used for NN model development under the Python environment. These libraries will automate many of the mathematical operations that we have derived and coded from scratch in Module 1. For example, the construction of the network, calculation of the loss function and its gradients, backpropagation using chain-rule, and the updating of weights and biases, all can be done automatically. Far from being wasted, the “maths” that we discussed in Module 1 allows us to understand what is actually happening behind these automated processes and avoid treating the application of AI as a “black box”.

3. PyTorch Workflow

For the rest of this module, we will be discussing the code implementation. It will be based on the workflow shown in Figure 1.

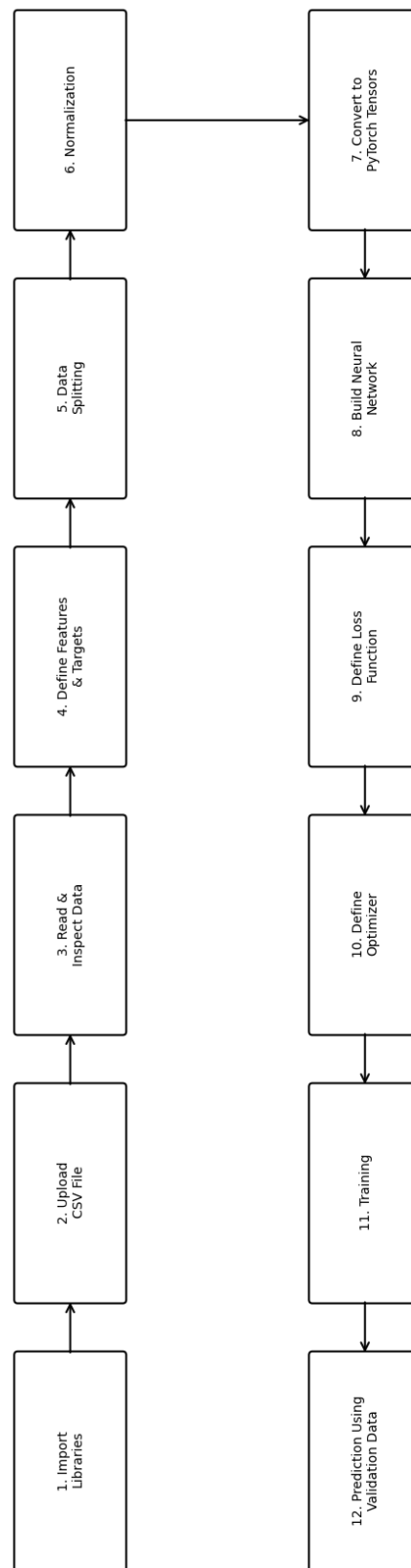
For the implementation, we will use Google Colab as our Python programming environment. Google Colab is suitable for workshops and beginners because we can write and run the Python code directly from the internet/web. Therefore, we don't have to spend much time on installing and setting up an Integrated Development Environment (IDE) and can focus more on the implementation of our NN.

However, just remember that the files uploaded into Google Colab are temporary and may be lost when the session ends or the runtime is restarted. Therefore, any important files during the session should be downloaded and saved.

Before we proceed, download the NN_simultaneous_equations.csv from:

https://msnm.com.my/wp/wp-content/uploads/2026/08/NN_simultaneous_equations.csv

Figure 1: PyTorch Workflow



4. The PyTorch Implementation (on Google Colab)

Step 1: Import libraries

The main libraries that we will use are:

- NumPy (for numerical and array operations)
- Pandas (for reading and handling our dataset)
- PyTorch (for building and training the NN)
- Matplotlib (for plotting and visualization)
- Scikit-learn (for data preprocessing and other ML procedures)

Copyable code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
```

Step 2: Upload CSV file

Before the data can be read and processed by our Python code, we first must upload the CSV file into our Google Colab. To do this, click the Files icon (folder icon) on the left panel of Google Colab. Then, click the Upload icon and select the file NN_simultaneous_equations.csv from your computer. Once uploaded, the file should appear in the Files panel. The CSV file is now available in the current Colab session and can be read by our Python code in the next step.

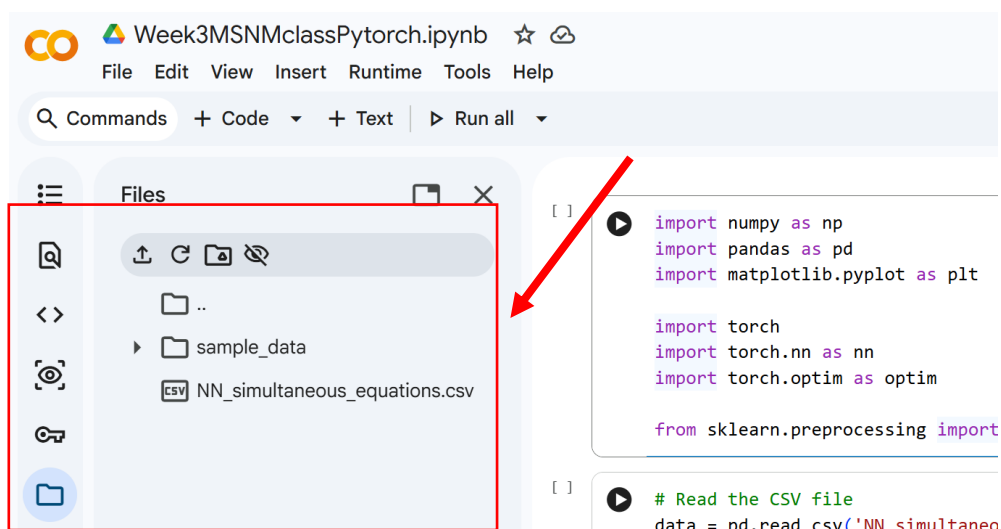


Figure 2: Uploading the CSV file

Step 3: Read and inspect data

To read a CSV file, we use the Pandas function `pd.read_csv()`. To inspect the data, we request to see the heading of the table in the DataFrame and the first 5 rows using function `print(data.head())`. We can also request the size of the data using function `print("\nDataset shape:", data.shape)`.

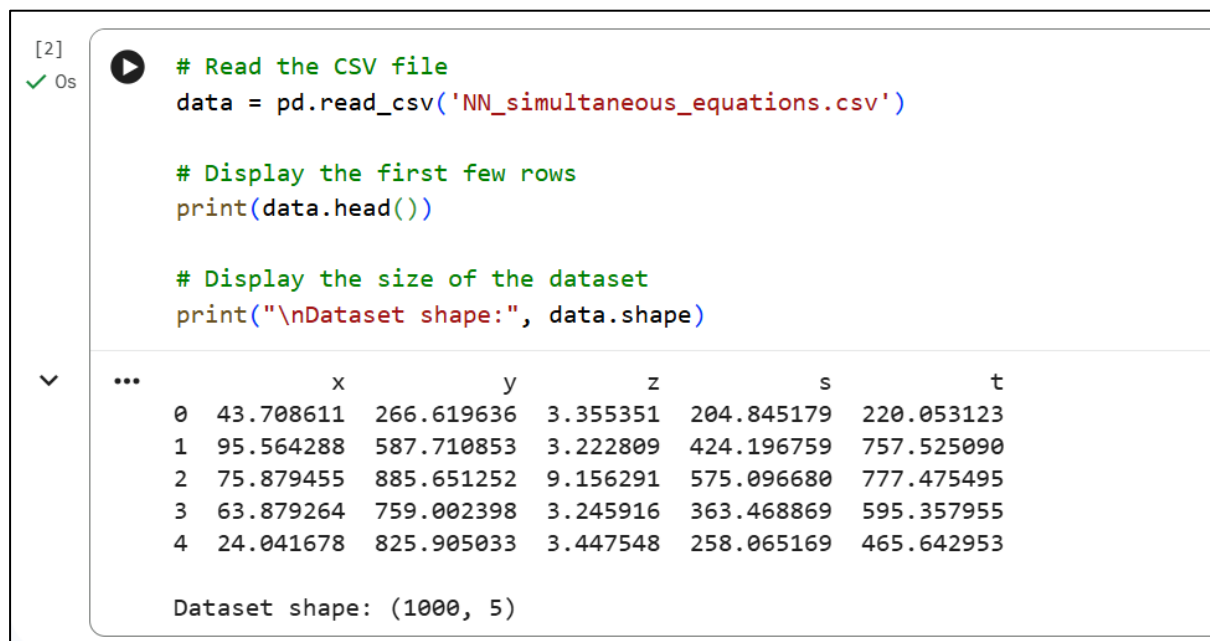
Copyable code:

```
data = pd.read_csv('NN_simultaneous_equations.csv')

print(data.head())

print("\nDataset shape:", data.shape)
```

If we run the code, we will see this output:



The screenshot shows a Jupyter Notebook cell with the following code and output:

```
[2]
✓ 0s # Read the CSV file
data = pd.read_csv('NN_simultaneous_equations.csv')

# Display the first few rows
print(data.head())

# Display the size of the dataset
print("\nDataset shape:", data.shape)
```

The output displays the first five rows of the DataFrame and the dataset shape:

	x	y	z	s	t
0	43.708611	266.619636	3.355351	204.845179	220.053123
1	95.564288	587.710853	3.222809	424.196759	757.525090
2	75.879455	885.651252	9.156291	575.096680	777.475495
3	63.879264	759.002398	3.245916	363.468869	595.357955
4	24.041678	825.905033	3.447548	258.065169	465.642953

Dataset shape: (1000, 5)

As we can see in Figure 3, the column names of the DataFrame are x, y, z, s, and t. This is a familiar scenery when we work in ML, where from here, we will identify the features (inputs) and the targets (outputs). For our problem, we know that x, y, and z are the inputs, hence the features, while s and t are the outputs, hence the targets.

The display of the first five instances (or samples) is very useful because it allows us to immediately check whether the values are as expected. For example, in our problem, we have described earlier the ranges for x, y, and z, and by looking at the display, we can see that they are within the expected ranges (i.e., $10 \leq x \leq 100$, $100 \leq y \leq 1000$, $1 \leq z \leq 10$).

The display of the size of the data (or shape) is also useful because it can confirm the expected size of our dataset. As mentioned earlier, we have generated 1000 labelled samples for this module. Therefore, we should expect the shape of our DataFrame to be (1000, 5).

Step 4: Define features and targets

From the inspection in Step 3, we know that x, y, and z are our features (inputs), while s and t are our targets (outputs). We will now separate these columns from the DataFrame and assign them to two variables, X and Y.

In Pandas, we can select the required columns from the DataFrame using `data[['column names']]`. We then use the `.values` attribute to return the selected values as a NumPy array. The NumPy array containing x, y, and z will be assigned to the variable X, while the NumPy array containing s and t will be assigned to the variable Y, as follows:

Features (input), X:

$$X = [x \ y \ z] \quad (5)$$

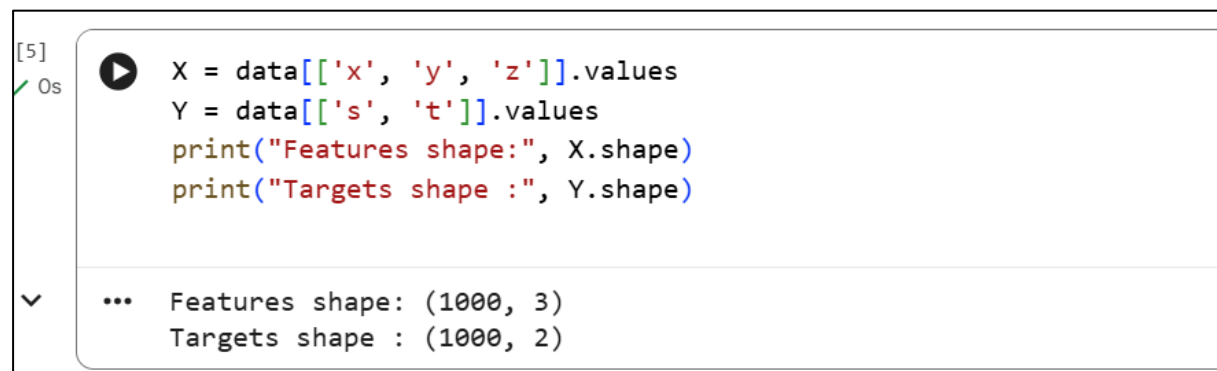
Targets (output), Y:

$$Y = [s \ t] \quad (6)$$

Copyable code:

```
X = data[['x', 'y', 'z']].values
Y = data[['s', 't']].values
print("Features shape:", X.shape)
print("Targets shape :", Y.shape)
```

If we run the code, we will see this output:

A screenshot of a Jupyter Notebook cell. The cell number is [5]. The code inside the cell is: `X = data[['x', 'y', 'z']].values`, `Y = data[['s', 't']].values`, `print("Features shape:", X.shape)`, and `print("Targets shape :", Y.shape)`. The output of the cell is: `... Features shape: (1000, 3)` and `Targets shape : (1000, 2)`.

```
[5] X = data[['x', 'y', 'z']].values
    Y = data[['s', 't']].values
    print("Features shape:", X.shape)
    print("Targets shape :", Y.shape)

... Features shape: (1000, 3)
Targets shape : (1000, 2)
```

From the printed shapes, we can see that the NumPy arrays have been created and their shapes are correct.

Step 5: Data splitting

As mentioned, since we are not discussing hyperparameter tuning in this module, we will split the data into 70% for training and 30% for validation. We split the data for each of the NumPy arrays (i.e., X and Y) so that they become X_train, Y_train, X_val, and Y_val.

To proceed with the splitting, we use `len(X)` to identify the total number of data to be split. Since one portion of the split is 70%, we multiply `len(X)` by 0.7. Then, the function `int()` is used to convert this value into an integer. The result is assigned to a variable, `split`.

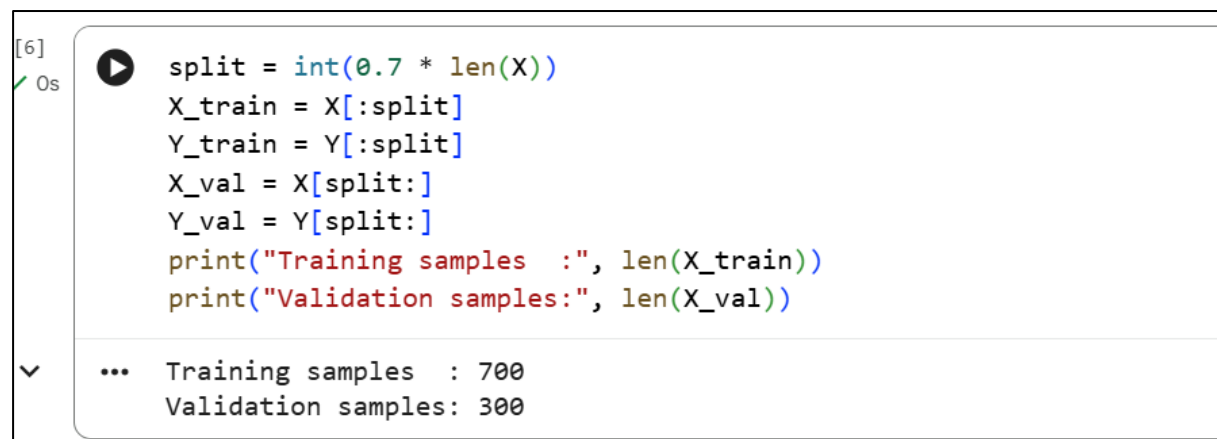
From this variable, we are then able to define, say, $X_{\text{train}} = X[:\text{split}]$, where it is an array of X from the first data (row 0) up to, but not including, the split index. With this definition, the remaining data, that is, from the split index onwards, are assigned to X_{val} .

Copyable code:

```
split = int(0.7 * len(X))
X_train = X[:split]
Y_train = Y[:split]
X_val = X[split:]
Y_val = Y[split:]

print("Training samples :", len(X_train))
print("Validation samples:", len(X_val))
```

If we run the code, we will see this output:

A screenshot of a Jupyter Notebook cell. On the left, there is a cell number [6] and a status icon. The main area contains Python code for splitting data. Below the code, the output is displayed, showing the number of training and validation samples. The code is:

```
split = int(0.7 * len(X))
X_train = X[:split]
Y_train = Y[:split]
X_val = X[split:]
Y_val = Y[split:]
print("Training samples :", len(X_train))
print("Validation samples:", len(X_val))
```

 The output is:

```
... Training samples : 700
Validation samples: 300
```

```
[6]
✓ Os
▶ split = int(0.7 * len(X))
  X_train = X[:split]
  Y_train = Y[:split]
  X_val = X[split:]
  Y_val = Y[split:]
  print("Training samples :", len(X_train))
  print("Validation samples:", len(X_val))

✓ ... Training samples : 700
  Validation samples: 300
```

From the printed lengths of X_{train} and X_{val} (i.e., $\text{len}(X_{\text{train}})$ and $\text{len}(X_{\text{val}})$), we can see that the splitting has been successful, where the sample sizes are appropriate.

Step 6: Normalization

As described earlier, normalization is needed to ensure that no variables with large values dominate the training and affect the learning process. The formula for normalization can be given as:

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (7)$$

where X_{norm} is the normalized value, X is the original value, $X_{\min}$ and $X_{\max}$ are the minimum and maximum values of the variable in the data, respectively. However, with the Scikit-learn library, the normalization process can be automated using the `MinMaxScaler` class.

The normalization is executed by the method `fit_transform()`. The method performs two processes. First, fit identifies the X_{min} and X_{max} values from the data. Then, transform uses these values to normalize the data. However, we only want to perform fit only to the training data. For the validation data, we want the normalization to be based on the X_{min} and X_{max} values of the `X_train` data (to prevent data leakage). Therefore, we perform `fit_transform()` to normalize `X_train` and only perform `transform()` to normalize `X_val`.

Copyable code:

```
scaler_X = MinMaxScaler()
scaler_Y = MinMaxScaler()
X_train_norm = scaler_X.fit_transform(X_train)
Y_train_norm = scaler_Y.fit_transform(Y_train)
X_val_norm = scaler_X.transform(X_val)
Y_val_norm = scaler_Y.transform(Y_val)
print("FEATURES BEFORE NORMALIZATION:")
print(X_train[:5])
print("\nFEATURES AFTER NORMALIZATION:")
print(X_train_norm[:5])
```

If we run the code, we will see this output:

```
[9]
✓ Os ▶ scaler_X = MinMaxScaler()
      scaler_Y = MinMaxScaler()
      X_train_norm = scaler_X.fit_transform(X_train)
      Y_train_norm = scaler_Y.fit_transform(Y_train)
      X_val_norm = scaler_X.transform(X_val)
      Y_val_norm = scaler_Y.transform(Y_val)
      print("FEATURES BEFORE NORMALIZATION:")
      print(X_train[:5])
      print("\nFEATURES AFTER NORMALIZATION:")
      print(X_train_norm[:5])

  ... FEATURES BEFORE NORMALIZATION:
    [[ 43.7086107  266.61963595  3.35535115]
     [ 95.56428758 587.71085264  3.22280919]
     [ 75.87945476 885.65125229  9.15629122]
     [ 63.87926358 759.00239777  3.2459158 ]
     [ 24.04167764 825.90503308  3.44754754]]

    FEATURES AFTER NORMALIZATION:
    [[0.3714636  0.18119427 0.26217763]
     [0.95073336 0.53994484 0.24741659]
     [0.73083789 0.87282933 0.90822136]
     [0.59678607 0.7313264  0.24998994]
     [0.15176809 0.80607575 0.27244543]]
```

From the printed values before and after normalization, we can see that the normalization has been successful, where none of the values after normalization are below zero or exceed one.

Step 7: Convert to PyTorch tensors

Up to Step 6, we have carried out the general processes that are common for most AI libraries. However, since we have decided to use PyTorch to build our NN architecture, we must convert our data into a format that can be read and handled by PyTorch. For this, we use the PyTorch function `torch.tensor()`. The resulting data are called PyTorch tensors. A tensor can be considered as a data structure used by PyTorch to store and perform operations on numerical data.

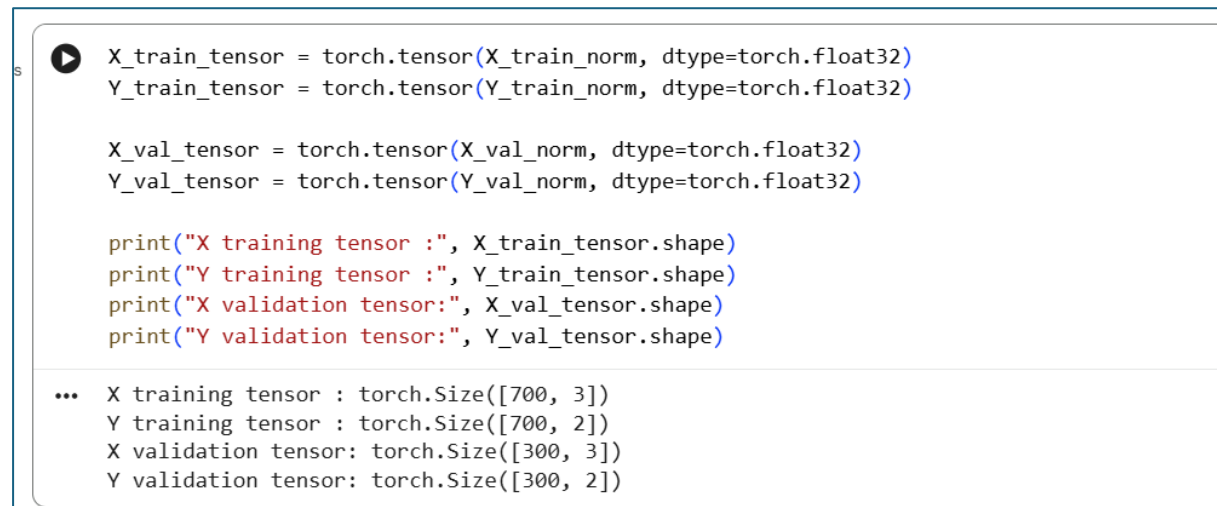
Copyable code:

```
X_train_tensor = torch.tensor(X_train_norm, dtype=torch.float32)
Y_train_tensor = torch.tensor(Y_train_norm, dtype=torch.float32)

X_val_tensor = torch.tensor(X_val_norm, dtype=torch.float32)
Y_val_tensor = torch.tensor(Y_val_norm, dtype=torch.float32)

print("X training tensor :", X_train_tensor.shape)
print("Y training tensor :", Y_train_tensor.shape)
print("X validation tensor:", X_val_tensor.shape)
print("Y validation tensor:", Y_val_tensor.shape)
```

If we run the code, we will see this output:



```
X_train_tensor = torch.tensor(X_train_norm, dtype=torch.float32)
Y_train_tensor = torch.tensor(Y_train_norm, dtype=torch.float32)

X_val_tensor = torch.tensor(X_val_norm, dtype=torch.float32)
Y_val_tensor = torch.tensor(Y_val_norm, dtype=torch.float32)

print("X training tensor :", X_train_tensor.shape)
print("Y training tensor :", Y_train_tensor.shape)
print("X validation tensor:", X_val_tensor.shape)
print("Y validation tensor:", Y_val_tensor.shape)

... X training tensor : torch.Size([700, 3])
    Y training tensor : torch.Size([700, 2])
    X validation tensor: torch.Size([300, 3])
    Y validation tensor: torch.Size([300, 2])
```

From the printed sizes of the tensors, we can see that the conversion has been successful, where the sample sizes are as expected.

Step 8: Build Neural Network

This is the crucial step where we decide the architecture of our network, and this involves deciding on:

- number of inputs and outputs
- number of hidden-layers and neuron per layer
- activation functions

To build our network, we start by using the PyTorch base class called `nn.Module`. This class provides the basic structure and functionalities for the purpose.

The initialization of the parameters (weights and biases) is done automatically when we create the model using `model = NeuralNetwork()`.

Copyable codes below provide two different architectures so that we can see the patterns of how can we decide and generate the above. The first code is for an architecture that has 4 inputs, 3 outputs, 2 hidden layers, 10 neurons per hidden layer and sigmoid activation functions, whilst the second code is for an architecture that has 3 inputs, 2 outputs, 3 hidden layers, 5 neurons per hidden layer and ReLU activation functions.

Copyable code 1 (4 inputs, 3 outputs, 2 hidden layers, 10 neurons per hidden layer and sigmoid activation functions):

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(4, 10)
        self.layer2 = nn.Linear(10, 10)
        self.layer3 = nn.Linear(10, 3)

    def forward(self, x):
        x = torch.sigmoid(self.layer1(x))
        x = torch.sigmoid(self.layer2(x))
        x = self.layer3(x)
        return x
model = NeuralNetwork()
print(model)
```

Copyable code 2 (3 inputs, 2 outputs, 3 hidden layers, 5 neurons per hidden layer and ReLU activation functions):

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(3, 5)
        self.layer2 = nn.Linear(5, 5)
        self.layer3 = nn.Linear(5, 5)
        self.layer4 = nn.Linear(5, 2)

    def forward(self, x):

        x = torch.tanh(self.layer1(x))
        x = torch.tanh(self.layer2(x))
        x = torch.tanh(self.layer3(x))
```

```
x = self.layer4(x)
return x
model = NeuralNetwork()
print(model)
```

Since our problem suits the number of inputs and outputs of the second copyable code, we will use and run that code. If we run the code, we will see this output:

```
91 class NeuralNetwork(nn.Module):
92     def __init__(self):
93         super().__init__()
94
95         self.layer1 = nn.Linear(3, 5)
96         self.layer2 = nn.Linear(5, 5)
97         self.layer3 = nn.Linear(5, 5)
98         self.layer4 = nn.Linear(5, 2)
99
100     def forward(self, x):
101
102         x = torch.tanh(self.layer1(x))
103         x = torch.tanh(self.layer2(x))
104         x = torch.tanh(self.layer3(x))
105         x = self.layer4(x)
106
107         return x
108
109 model = NeuralNetwork()
110 print(model)
```

```
... NeuralNetwork(
  (layer1): Linear(in_features=3, out_features=5, bias=True)
  (layer2): Linear(in_features=5, out_features=5, bias=True)
  (layer3): Linear(in_features=5, out_features=5, bias=True)
  (layer4): Linear(in_features=5, out_features=2, bias=True)
)
```

From the printed outputs, we can see that our NN architecture has been generated. This architecture is shown graphically in Figure 3 below.

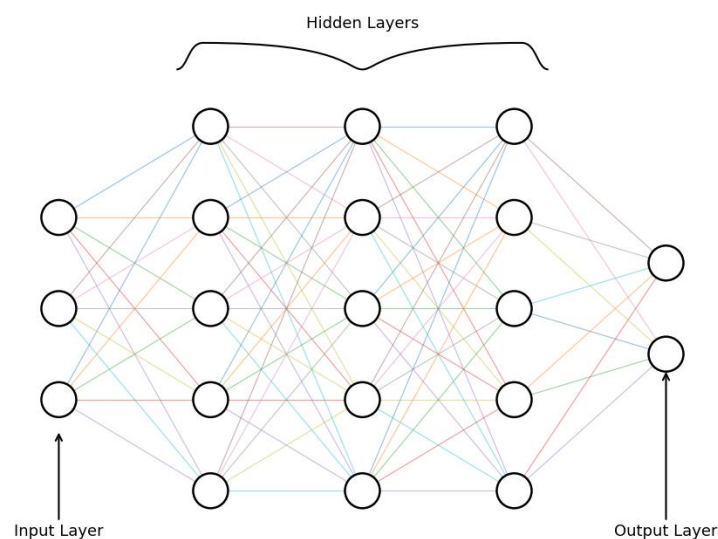


Figure 3: NN architecture generated by the second copyable code

Step 9: Define Loss function

Defining the loss functions depends on the type of problem (e.g., regression or classification). Since ours is a regression problem, some of the loss functions relevant to us are:

- i. squared error or mean-squared error (MSE)
- ii. mean-absolute error (MAE)
- iii. Huber loss

In this module, we select MSE as our loss function, thus we employ `criterion = nn.MSELoss()`. But to swap to another loss function, say MAE, we simply employ `criterion = nn.L1Loss()`, as that's how PyTorch implement MAE. For Huber loss, the implementation is `nn.HuberLoss()`.

Copyable code:

```
criterion = nn.MSELoss()
```

Run the code, but don't expect any output because we do not ask the code to print or display anything.

Step 10: Define optimizer

Optimizers are the algorithms that carry out the updating of the parameters (weights and biases). The process involves the calculation of the gradient of the loss with respect to the parameters (i.e., $\frac{\partial Loss}{\partial w_{ij}}$) by chain-rule, which will be automated by PyTorch (using `loss.backward()`), to be discussed in Step 11), and the specification of the learning rate, μ , which usually decided by "trial and error" or experiment.

In Module 1, we have discussed the optimization by Gradient Descent (GD) or Stochastic Gradient Descent (SGD) as:

$$w_{new} = w_{old} - \mu \frac{\partial Loss}{\partial w_{old}} \quad (6)$$

However, with PyTorch, there are other optimizers available such as Adam, Adagrad, RMSProp, LBFGS and many other variations. In this module, we opt for SGD as optimizer by employing `optim.SGD(model.parameters(), lr=learning_rate)`.

But to swap to Adam, or Adagrad or RMSProp, or LBFGS, we simply employ `optim.Adam()`, `optim.Adagrad()`, `optim.RMSprop()`, or `optim.LBFGS()`, respectively.

To note, argument `model.parameters()` refers to all the trainable parameters (weights and biases) of our NN.

Copyable code:

```
learning_rate = 0.01  
  
optimizer = optim.SGD(model.parameters(), lr=learning_rate)
```

Run the code, but don't expect any output because we do not ask the code to print or display anything.

Step 11: Training

So far, we have prepared all we need to setup a NN architecture, but we haven't executed the training yet. To execute the training means we need to instruct the following process:

- i. the forward pass
- ii. the loss calculation
- iii. the loss gradient calculation
- iv. the parameter update

Since training will involve looping, the main measure is called an epoch. One epoch is defined as one complete cycle where all the training data/samples have been used for the training. In our problem, since all 700 training samples are considered as one batch, 1 epoch is completed when all the 700 samples have been utilized for the training and parameter update.

And to monitor the progress of the training (where we would expect a convergence), we would record the calculated loss over the epochs.

Copyable code (the training without convergence visualisation):

```
epochs = 5000 # Number of epochs
loss_history = [] # To record the loss

for epoch in range(epochs):

    Y_pred = model(X_train_tensor) # Forward pass

    loss = criterion(Y_pred, Y_train_tensor) # Calculate loss

    optimizer.zero_grad() # Clear old gradients

    loss.backward() # Backpropagation

    optimizer.step() # Update weights and biases

    loss_history.append(loss.item()) # Record loss
```

Run the code above if we just want to carry out the parameter updates. But if we want to have a visualisation for monitoring, run the copyable code below, as it will give us a real-time plot of the loss convergence curve over the epochs.

Copyable code (the training with real-time convergence visualisation):

```
from IPython.display import clear_output
import time

epochs = 5000 # Number of epochs
loss_history = [] # To record the loss

for epoch in range(epochs):
```



```
Y_pred = model(X_train_tensor) # Forward pass

loss = criterion(Y_pred, Y_train_tensor) #Calculate loss

optimizer.zero_grad() # Clear old gradients

loss.backward() # Backpropagation

optimizer.step() # Update weights and biases

loss_history.append(loss.item())# Record loss

if (epoch + 1) % 50 == 0:

    clear_output(wait=True)

    plt.figure(figsize=(8, 5))
    plt.plot(loss_history)

    plt.xlabel('Epoch')
    plt.ylabel('MSE Loss')
    plt.title('Neural Network Training Loss')

    plt.grid()
    plt.show()
```

By running the second copyable code, we would obtain the convergence curve as shown in Figure 4. As can be seen, the loss reduces as the number of epochs increases. This indicates that loss is being minimized and the errors are diminishing, hence showing convergence.

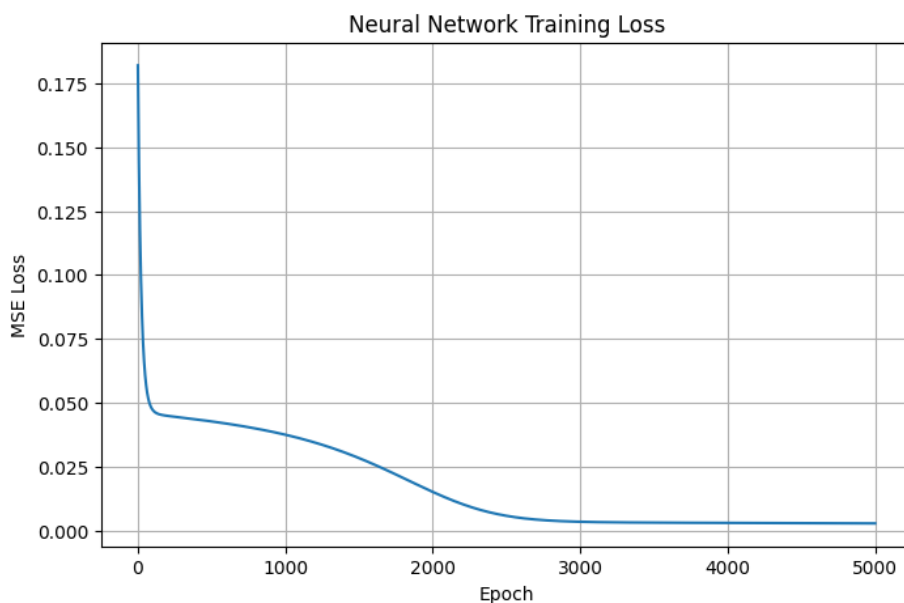


Figure 4: Convergence of the Loss

Step 12: Prediction using validation data

Now that we have completed the training, it's time to check whether our model can really make predictions. For this, we will use the remaining 30% of the dataset, which we have specified as `X_val` and `Y_val` and later transformed into PyTorch tensors, denoted as `X_val_tensor` and `Y_val_tensor`, respectively.

To make the prediction, we use `model.eval()` together with `torch.no_grad()` to tell PyTorch that we are not training anymore (thus, we don't need the gradients).

To make the predictions, we use `X_val_tensor`, which is in the normalized form. Therefore, the predicted output is also in the normalized form and returned as a PyTorch tensor, denoted as `Y_val_pred_norm`.

Therefore, to make the comparison between the actual and predicted values physically meaningful, we need to inverse-transform `Y_val_pred_norm` back to its original magnitude (denoted as `Y_val_pred`). We don't need to inverse-transform `Y_val` because it is already in its original magnitude.

So, the comparison will be `Y_val_pred` vs `Y_val`.

Copyable code (for prediction):

```
model.eval()
with torch.no_grad():
    Y_val_pred_norm = model(X_val_tensor)

Y_val_pred_norm = Y_val_pred_norm.numpy()

Y_val_pred = scaler_Y.inverse_transform(Y_val_pred_norm)
```

Now, the code above is about calculating the predicted values of the output, we are yet to compare them against the actual values. To make such comparison, we can either calculate the difference between `Y_val_pred` and `Y_val` (in the forms of performance metric, MSE, RMSE, MAE) or create a scatter plot where the coordinates of each data point are (`Y_val`, `Y_val_pred`) and assess how far the points are from the 45-degree reference line.

A quick check on the model performance can be made by looking at the MAE and RMSE, where relatively small values indicate a better-performing model. Also, a good-performing model would have the scattered data points very close or in proximity to the reference line.

Copyable code: Calculating performance (MSE, RMSE, MAE)

```
actual_s = Y_val[:, 0]
predicted_s = Y_val_pred[:, 0]

actual_t = Y_val[:, 1]
predicted_t = Y_val_pred[:, 1]

# Performance metrics for s
MSE_s = mean_squared_error(actual_s, predicted_s)
RMSE_s = np.sqrt(MSE_s)
```

```

MAE_s = mean_absolute_error(actual_s, predicted_s)

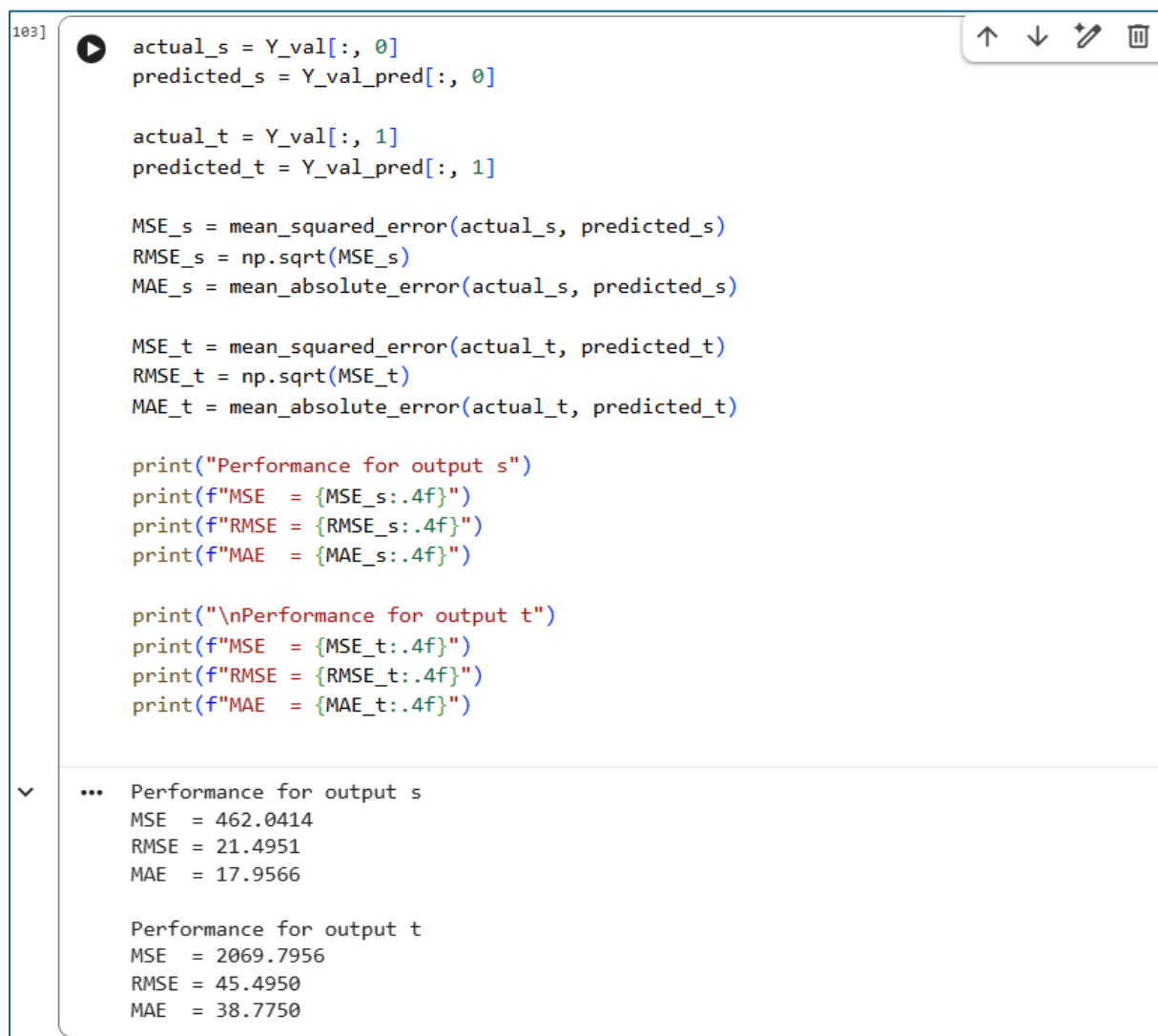
# Performance metrics for t
MSE_t = mean_squared_error(actual_t, predicted_t)
RMSE_t = np.sqrt(MSE_t)
MAE_t = mean_absolute_error(actual_t, predicted_t)

# Display results
print("Performance for output s")
print(f"MSE = {MSE_s:.4f}")
print(f"RMSE = {RMSE_s:.4f}")
print(f"MAE = {MAE_s:.4f}")

print("\nPerformance for output t")
print(f"MSE = {MSE_t:.4f}")
print(f"RMSE = {RMSE_t:.4f}")
print(f"MAE = {MAE_t:.4f}")

```

If we run the code, we can see this output:



The screenshot shows a Jupyter Notebook cell with the following code and output:

```

103] actual_s = Y_val[:, 0]
      predicted_s = Y_val_pred[:, 0]

      actual_t = Y_val[:, 1]
      predicted_t = Y_val_pred[:, 1]

      MSE_s = mean_squared_error(actual_s, predicted_s)
      RMSE_s = np.sqrt(MSE_s)
      MAE_s = mean_absolute_error(actual_s, predicted_s)

      MSE_t = mean_squared_error(actual_t, predicted_t)
      RMSE_t = np.sqrt(MSE_t)
      MAE_t = mean_absolute_error(actual_t, predicted_t)

      print("Performance for output s")
      print(f"MSE = {MSE_s:.4f}")
      print(f"RMSE = {RMSE_s:.4f}")
      print(f"MAE = {MAE_s:.4f}")

      print("\nPerformance for output t")
      print(f"MSE = {MSE_t:.4f}")
      print(f"RMSE = {RMSE_t:.4f}")
      print(f"MAE = {MAE_t:.4f}")

```

The output of the code is as follows:

```

... Performance for output s
MSE = 462.0414
RMSE = 21.4951
MAE = 17.9566

Performance for output t
MSE = 2069.7956
RMSE = 45.4950
MAE = 38.7750

```

From the output, we can see the performance metrics for s and t , respectively. As mentioned, a quick check on the model performance can be made by looking at the MAE and RMSE, where relatively small values indicate a better-performing model. However, just from one run, it is quite difficult to determine whether the metrics are “small” enough, hence the importance of the stage of hyperparameter tuning, which we will discuss in greater detail in future modules.

There is a possibility that by increasing the number of layers, changing the number of neurons per layer, opting for more suitable activation functions and optimizers, experimenting with the learning rates, etc., we can obtain smaller values of the performance metrics, hence better accuracy of the predictions. These are some of the options that can be explored during hyperparameter tuning.

For now, especially based on the scatter plots (shown next), we can say that our model has successfully represented Eqs. (3) and (4), where the accuracy can be further enhanced through hyperparameter tuning. Thus, we have attained the objective of this module.

Copyable code: Scatter plots

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots
min_s = min(actual_s.min(), predicted_s.min())
max_s = max(actual_s.max(), predicted_s.max())
min_t = min(actual_t.min(), predicted_t.min())
max_t = max(actual_t.max(), predicted_t.max())
fig = make_subplots(
    rows=1,
    cols=2,
    subplot_titles=("Output s", "Output t")
)
fig.add_trace(
    go.Scatter(
        x=actual_s,
        y=predicted_s,
        mode='markers',
        name='Prediction s',
        hovertemplate=
            'Actual s = %{x:.2f}<br>' +
            'Predicted s = %{y:.2f}<extra></extra>'
    ),
    row=1,
    col=1
)

fig.add_trace(
    go.Scatter(
        x=[min_s, max_s],
        y=[min_s, max_s],
        mode='lines',
        name='45° Reference Line'
    ),
    row=1,
    col=1
)
```

```

)
fig.add_trace(
    go.Scatter(
        x=actual_t,
        y=predicted_t,
        mode='markers',
        name='Prediction t',
        hovertemplate=
            'Actual t = %{x:.2f}<br>' +
            'Predicted t = %{y:.2f}<extra></extra>'
    ),
    row=1,
    col=2
)

fig.add_trace(
    go.Scatter(
        x=[min_t, max_t],
        y=[min_t, max_t],
        mode='lines',
        name='45° Reference Line',
        showlegend=False
    ),
    row=1,
    col=2
)
fig.update_xaxes(title_text='Actual s', row=1, col=1)
fig.update_yaxes(title_text='Predicted s', row=1, col=1)

fig.update_xaxes(title_text='Actual t', row=1, col=2)
fig.update_yaxes(title_text='Predicted t', row=1, col=2)
fig.update_yaxes(scaleanchor='x', scaleratio=1, row=1, col=1)
fig.update_yaxes(scaleanchor='x2', scaleratio=1, row=1, col=2)
fig.update_layout(
    title='Actual vs Predicted Values',
    width=1100,
    height=500
)
fig.show()

```

If we run the code above, we would obtain the scatter plots shown in Figure 5. Based on the figure, we can see that our model is somehow able to extract the mathematical patterns of Eqs. (3) and (4), where the data points are scattered quite close to the reference line (which is supposed to be where the predictions are exactly correct). This again highlights the attainment of the objective of this module. And again, there is a possibility that hyperparameter tuning will bring the data points closer or “tighter” to the reference line.

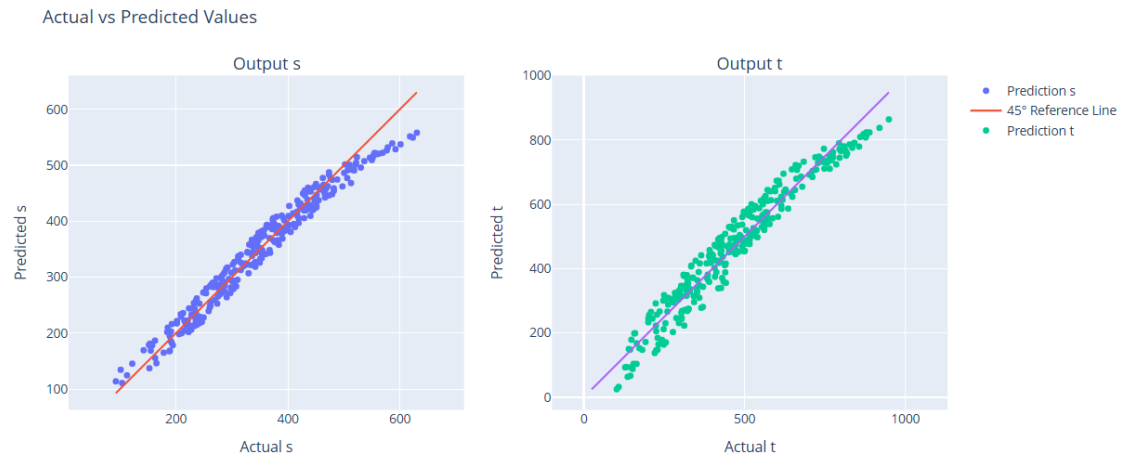


Figure 5: Scatter plots for the predictions of s and t from Eqs. (3) and (4)